

OOP-3. 행동

3. 행동

객체는 데이터로서 존재하는 것이 아니라 **"행동"** 하는 것이 중요하다.
가장 쉽게 객체를 행동하게 만드는 방법은 **TDA 원칙** 적용하는 것이다.

TDA 원칙은 객체를 행동하게 만든다. 그리고 TDA 원칙이 객체를 행동하게 만든다고 말하는 이유는 간단하다.

"물지 말고 시켜라" 라는 말 자체가 객체에 어떤 행동을 하라고 조언하는 것이기 때문이다.

하지만 TDA 원칙은 속성을 이미 갖고 있는 객체가 있을 때 사용하기 좋으며 수동적인 객체를 능동적인 객체로 바꾸는 방법이다.

즉, 이미 존재한다는 것을 전제로 할 수 있는 원칙이다.

그러면 객체를 설계하는 단계라면 어떻게 행동에 집중하는 객체를 만들 수 있을까?

사고 방식의 차이

다음과 같은 요청이 들어온다면 이를 어떻게 구현할까?

 "자동차 클래스를 만들어 줄 수 있나요?"

개발자 A의 접근법

자동차에 필요한 부품과 구현에 필요할 것으로 예상되는 속성들을 떠올린다.

```
public class Car {  
    private Frame frame;  
    private Engine engine;  
    private List<Wheel> wheels;  
    private float speed;  
    private float direction;  
}
```

개발자 B의 접근법

필요한 행동을 먼저 떠올린다.

```
public class Car {  
    public void drive() {}  
    public void changeDirection(float amount){}  
    public void accelerate(float speed){}  
    public void decelerate(float speed){}  
}
```

우선 두 코드가 얼마나 설계적으로 말이 되고 적합한지는 무시한다.
여기서 두 개발자가 어떻게 다르게 생각했고 어떤 결과물이 만들어졌는지 이해해야 한다.

개발자 A는 **데이터 위주의 사고**를 하였지만 개발자 B는 **행동 위주의 사고**를 했다.
그리고 각자 다른 결과가 나오게 되었다.

두 접근법의 비교

데이터 위주의 사고 방식으로 만들어진 이 클래스에는 필요한 속성이 정의돼 있다.
어떤 객체를 정의하는 데 필요한 속성으로만 묶여 있다.
이러한 클래스는 절차지향 언어에서 구조적인 데이터 덩어리를 만드는 데 사용하는 **"구조체"**와 다를 바 없다.
따라서 전혀 **"객체지향"**스럽지 않다.

행동 위주의 사고 방식으로 만들어진 클래스는 벌써부터 객체가 어떤 동작을 하는지 알 수 있다.
이 클래스의 객체가 어디에 어떻게 쓰일지 눈에 보인다는 것이다.

물론 뭐가 더 좋은 코드라고 단정 지을 수 없다. 클래스를 만드는 목적에는 데이터를 구조적으로 만든다는 것도 포함되기 때문이다.

하지만 **객체지향을 논할 때는 개발자 B의 코드가 더 나은 코드이다.**

왜냐하면 **"객체지향 관점에서 객체는 서로 협력해야 하기 때문이다."**

"협력"을 하려면 상대방에게 **"행동"**을 요구할 수 있어야 한다.

따라서 행동위주의 사고를 하는 편이 객체지향에서는 훨씬 유리하다고 볼 수 있는 것이다.

핵심 원칙

💡 객체를 구분 짓는 요인은 데이터가 아닌 **"행동"**이다.

객체를 만들 때 데이터보다 행동에 집중해야 하며 데이터가 객체를 결정하지 않는다.
"행동이 객체를 결정한다."

이름 짓기 실험

다음 코드의 클래스는 어떤 이름으로 짓는 것이 적절할까?

```
public class ??? {  
    private float speed;  
    private float direction;  
}
```

지금까지 계속 자동차 예시를 들어왔으니 Car 라고 대답할지도 모른다.
하지만 냉정하게 생각해보면 이 클래스의 이름을 짓는 것은 매우 어렵다.

속도와 방향을 가지고 있는 개체는 너무나도 많기 때문이다.

반면 다음과 같은 클래스가 있다고 하면 어떤 이름을 지을 수 있을까?

```
public class ??? {  
    public void ride(){ ... }  
    public void run() { ... }  
    public void stop() { ... }  
}
```

탐승하며 달릴 수 있으니 '탈 것'을 뜻하는 `Vehicle` 라고 이름을 지을 수 있다.

이처럼 객체가 갖고 있는 속성이나 세부 구현에 집중하기보다 **행동에 집중할 때 더 자연스러운 모델링이 될 수 있다.**

데이터로는 객체를 정의하기 어렵지만 행동을 보면 객체를 정의하기 쉽기 때문이다.

행동과 역할

행동을 먼저 고민 했더니 구체적인 이름보다 '탈 것' 이라는 역할이 나왔다.

행동을 고민하는 것은 자연스럽게 역할을 고민하게 만든다.

따라서 어떤 행동을 할 수 있는지가 곧 역할을 만든다.

이러한 여러 역할이 모여 객체를 정의하는 것이다.

3.1 덕 타이핑

객체지향은 행동을 강조한다.

그리고 이 사실을 뒷받침하는 데 사용하기 좋은 용어가 있다.

바로 "**덕 타이핑**" 이다. 덕 타이핑의 개념은 덕 테스트에서 유래했다.

 **덕 테스트:** 만약 어떤 새가 오리처럼 걷고, 헤엄치고, 깉깉거리는 소리를 낸다면 나는 그 새를 오리라고 부를 것이다.

개발자 관점에서 이 말을 해석하자면, "**행동이 같다면 같은 클래스**" 로 부르겠다는 의미이다.

하지만 이 말을 처음 들으면 크게 와 닿지 않을 수 있다. 왜냐하면 자바 같은 정적 타입 언어를 다루고 있는 입장에서는

이 말을 쉽게 체감할 수 없기 때문이다.

그래서 자바와 최대한 비슷한데 덕 타이핑을 지원하는 대표적인 언어 **타입스크립트**를 이용해 이게 무슨 뜻인지 확인해보겠다.

```
class Duck{  
    walk = (): void => {  
        ...  
    }  
  
    swim = ():void => {
```

```

    ...
}

quackQuack = (): void => {
    ...
}
}

```

이어서 UnknownBird 라는 클래스도 만들었다.

```

class UnknownBird {

    age: number;

    walk = (): void => {
        ...
    }

    swim = ():void => {
        ...
    }

    quackQuack = (): void => {
        ...
    }
}

```

문법적으로 이 코드가 어떤지는 굳이 설명하지 않아도 이해할 수 있을 것이다.

Duck 과 UnknownBird 클래스는 walk , swim , quackQuack 메소드를 갖고 있고 UnknownBird 는 Duck 에는 없는 age 라는 멤버 변수가 있다.

우선 **자바 개발자 관점에서는 두 클래스는 완전히 다른 클래스이다.**

왜냐하면 두 클래스 사이에 어떠한 상속 관계도 없기 때문이다.

그리고 UnknownBird 에는 age 같은 Duck 클래스에는 없는 속성이 들어 있다.

따라서 **데이터 관점에서도 두 클래스는 완전히 다르다고 볼 수 있다.**

하지만 타입스크립트에서는 다음과 같이 상속 관계를 맺지 않은 객체를 할당할 수 있다.

```

const duck: Duck = new UnknownBird();

```

즉, Duck 타입 변수인 duck 에 UnknownBird 타입의 객체를 생성해서 할당할 수 있다.

이것이 가능한 이유는 **타입스크립트가 행동이 같은 두 클래스를 같은 클래스로 보겠다는 덕 타이핑 개념을 지원하기 때문이다.**

UnknownBird 가 어떤 새인지 모르지만 Duck 처럼 행동하므로 그냥 오리라고 봐도 된다는 것이다.

자바 개발자의 상식에서 벗어나 **행동이 같으면 같은 클래스로 보겠다**라는 덕 타이핑 개념이 다른 언어에서는

어떻게 구현됐는지를 보고 확인하면 좋을 것 같다.

⚙️ 3.2 행동과 구현

행동이 중요하고 행동 위주의 사고를 해야 한다는 사실을 우리는 알게되었다.

하지만 개발자 B의 코드가 이제 끝난 것일까? 과연 정말로 괜찮은 코드일까?

아직 아니다. 조금만 더 고민해볼 필요가 있다.

우선 개발자 B의 코드는 **미완성된 코드이다**. 메소드의 구현이 없기 때문이다.

그래서 비교적 간단해 보이는 `changeDirection` 메소드를 구현해보았다.

이 메소드는 방향을 바꾸는 메소드이며 이 메소드를 호출하면 그 결과로 자동차의 각도를 변경한다.

변경 가능한 각도는 0~360도이고 북쪽을 기준으로 얼마만큼 시계방향으로 기울어져 있는지를 나타낸다.

그리고 각도를 0~360도로 유지해야하므로 지정된 값을 벗어날때 값을 재조정 하도록 만든다.

```
public class Car {
    private int degree; //자동차의 각도 (0~360도)

    public void drive(){}
    public void changeDirection(float amount){
        float result = (degree + amount) % 360;
        if(result < 0 ) result += 360;
        this.degree = (int) result;
    }

    public void accelerate(float speed){}
    public void decelerate(float speed){}
}
```

`Car` 클래스에 속성이 생겼다.

메소드를 구현하려고 했더니 데이터 위주의 사고로 돌아온 것이다.

여기서 우리는 한 가지 사실을 알 수 있다.

행동의 구현을 고민했더니 결국 이 클래스가 어떤 값을 갖고 있어야 하는지를 고민하게 됐다는 것이다.

그리고 이는 그다지 반가운 현상은 아니다.

기껏 행동 위주로 사고하겠다고 다짐했었는데 구현을 고민했더니 곧바로 데이터 위주의 사고로 되돌아왔다.

⚠️ 문제의 원인

이러한 현상은 **구현을 고민했기에 나타는 현상이다.**

행동을 고민하면서 구현이나 알고리즘을 고민해서는 안된다.

행동을 고민하는 순간에는 이 클래스에 어떤 동작을 시킬 수 있을 것인지만 고민하는 것이 좋다.

오롯이 **어떤 메시지를 처리할지만 고민하는 것이다.** 어떻게는 그 다음 문제이다.

💡 해결책: 인터페이스 활용

그러면 어떻게 해야 구현에 구애받지 않고 행동만 고민할 수 있을까?

메소드의 본문을 그냥 비워두기만 할까? 그런데 반환형이 있는 메소드라면 어떡할까?

반환 값이 없다면 컴파일 에러가 날 것이다. 그냥 임시로 아무 데이터나 반환하도록 만들어둘까?
하지만 그렇게 만들면 나중에 이 코드를 다시 봤을 때 해당 코드가 개발이 완료된 것인지 아닌지 헷갈릴 것이다.

그럴 때 사용하는 것이 인터페이스이다.

자바의 인터페이스를 활용하면 구현 없이도 메소드를 정의할 수 있다.

또한 인터페이스는 구현을 가질 수 없게 강제해 놓았다.

의문점과 답변

여기서 의문점이 생긴다.

과연 초기 설계 단계에서는 상세한 구현은 무시해도 괜찮을까?

이에 대답은 **초기 설계 단계에서는 상세한 구현은 무시해도 괜찮다.**

구현에 관해 욕심을 부리기 시작하면 결국 처음부터 끝까지 모든 프로그램을 다 만들어야한다.

시스템을 개발하는 초기에 팀원과 모두 모여 **역할과 행동을 기반으로 인터페이스를 정의하는 것이다.**

그 다음 **객체들이 어떻게 협력할지 합의한다.**

모든 합의가 끝나면 인터페이스를 이용해 개략적인 코드가 나올 수 있다.

이러한 방법은 **인터페이스를 제대로 지켜야하며**, 인터페이스를 기반으로 협력방식을 정해놓았기 때문에 지켜야한다.

이럴 때는 이해 관계자들이 모여 설계를 다시 해야하며, **인터페이스는 곧 계약이다** 같은 말이 있는 것이다.

3.3 인터페이스

앞서 말한 부분에서 오해가 생길 수 있다.

인터페이스와 행동은 다르다는 것이다.

행동에 대한 말을 하다보니 인터페이스로 이어졌지만, **행동이 곧 인터페이스는 아니다.**

이 둘을 구분할 필요가 있다.

인터페이스는 외부에서 어떤 객체에게 행동을 시키고자 할 때 메시지를 보낼 수 있는 창구일 뿐이다.

다시말해, **어떤 행동을 지시하는 방법의 집합이라고 표현할 수 있다.**

 **인터페이스란**, 나를 조작하고 싶다면 이런 메시지를 보내면 된다고 외부에 알려주는 수단이다.

더불어 인터페이스의 목적이 외부세계에 나를 다루는 방법을 알려주는 것이므로 별도의 지시자가 없이도 `public` 으로 동작한다.

객체들이 인터페이스를 통해 통신하게 되면 객체 간의 결합도를 낮출 수 있다.

그 결과, **유연성과 확장성을 얻을 수 있다.**

인터페이스를 사용하는 코드는 **재사용도 높아지고, 유지보수성도 올라간다.** 또한 **모듈화도 가능하다.**

이외에도 인터페이스를 사용하는 코드의 이론적인 장점을 더 나열할 수도 있다.

3.4 메소드

메소드는 왜 메소드라고 부르고, 메소드와 함수에는 어떠한 차이가 있는 것일까?

함수가 클래스 안으로 들어온다고 해서 메소드가 되는 이유가 무엇일까?

인터페이스의 정의를 다시보자.

 **인터페이스란** 나를 조작하고 싶다면 이런 메시지를 보내면 된다고 외부에 알려주는 수단이다.

즉, 어떤 객체나 시스템을 다루는 외부 세계는 **협력 대상과 소통하기 위해 메시지를 이용한다.**

객체는 **협력 객체에 메시지를 건내고, 협력 객체는 메시지를 수신해서 행동을 하는 것이다.**

우리는 어떤 객체가 협력 객체에 요청을 보낼 때 어떤 특정 메소드나 함수를 실행한다고 생각한다.

하지만 사실 그렇지 않다.

실제로는 인터페이스를 이용한 통신에서는 메시지를 보내면서도 어떤 메소드가 실행될지는 모른다.

```
class Car implements Vehicle {
    void ride() {}
    void run() {}
    void stop() {}
}

class Bicycle implements Vehicle {
    void ride() {}
    void run() {}
    void stop() {}
}

class User {

    void ride(Vehicle vehicle) {
        //User 클래스는 어떤 객체의 메소드를 실행하게 될까요?
        // Car 객체의 ride() 메소드? / Bicycle 객체의 ride() 메소드?
        //실제로 코드가 실행되기 전까지는 알 수 없다.
        vehicle.ride();
    }
}
```

vehicle.ride() 호출하지만, 코드가 실행되기 전까지는 실제로 어떤 메소드가 호출될지 모른다.

이 같은 당연한 사실 때문에 메소드와 함수는 다른 개념이 된다.

함수 개념으로는 메소드를 정의할 수 없기 때문이다.

 **함수는 입력값과 출력값 사이의 대응 관계를 나타낸다.**

함수의 각 입력값은 정확히 하나의 출력값으로 대응된다.

여기서 주목해야할 점은 **함수의 각 입력값은 정확히 하나의 출력값으로 대응된다**

함수는 같은 입력에 대해 항상 같은 출력을 해야한다.

같은 입력값으로 함수를 실행했을 때 다른 출력 값이 나와서는 안된다.

즉, **같은 입력에 대해 두 개의 출력을 갖는 함수가 있어서는 안된다.**

함수를 실행하면서도 실제로 어떤 함수가 실행될지 모른다는 말은 부적절하다.

함수를 실행한다는 것은 고정적이고 특정하게 지정된 절차를 수행한다는 의미라서 그렇다.

따라서 어떤 클래스의 어떤 메소드가 실행될지 모르면서 함수를 실행한다고 표현하는 것은 불가능하다.
그리고 이는 객체지향에서 추구하는 방향과 반대가 된다.

객체지향에서는 특정한 구현에 의존하는 상황을 피하고자한다.

그래서 객체지향에서는 협력 객체에 어떤 일을 요청할 때 함수를 실행한다는 말 대신 메시지를 전달한다고 표현한다.

객체지향에서 객체들은 메시지를 통해 소통한다.

이러이러한 인수를 건네줄테니 알아서 일을 해달라고 부탁하는 것이다.

즉, 객체는 협력 객체에 메시지만 보낼 뿐이다.

실제로는 어떤 방법(method) 으로 일을 어떻게 처리할지는 객체가 정한다.

그래서 객체지향에서는 객체가 수행하는 함수를 메소드라고 부른다.

따라서 메소드를 어떻게 구현할지에 집중하는 것은 그다지 좋은 방법은 아니다.

메소드란, 결국 어떤 메시지를 어떻게 처리하는지를 서술하는 것이므로 알고리즘에 가까운 것이기 때문이다.

알고리즘을 고민하면 결국 구현에 집착하게 된다.

결국 메소드 작성 행위가 함수를 작성하는 것과 다를 바 없어지고 절차지향적 코드가 나오게 되는 것이다.